

Quantitative Engineering Architecture Audit: Indian Equities Framework

1. Executive Summary

Forensic analysis of the Research Corpus v1 reveals a deeply researched, multi-domain framework designed for algorithmic swing trading in Indian equities. The corpus correctly identifies that systemic risk, data corruption, and concept drift are the primary destroyers of quantitative capital. The architecture mandates a strict delineation between probabilistic AI (used exclusively for semantic reasoning and sentiment analysis) and deterministic Python execution (used for signal generation and trade routing).

However, critical gaps exist. The corpus assumes stable API access during tail-risk events, relies heavily on third-party APIs (Upstox, Angel One) that have documented historical gaps, and proposes hybrid AI-assisted signal weighting schemas without providing the mathematical loss functions required to prevent AI model overfitting.

Final Assessment: The foundational architecture (DuckDB/SQLite, local-first, deterministic execution) is **Supported** and highly robust. The integration of LLMs for sentiment analysis is **Partially Supported**, carrying high integration risk.

2. Corpus Reliability Audit

Document	Authority Level	Evidence Type	Bias / Limitation
AI_Assisted_Swing_Trading_Architecture	High	Architectural heuristics	Highly prescriptive; assumes local-first viability at scale
High_Impact_News_Signals	High	Empirical / Academic	Heavily reliant on US-centric academic studies; Indian market parallels assumed
Indian_Equities_Data_Source_Ranking	Medium-High	Practitioner testing	Survivorship bias in API evaluations; API endpoints frequently change
Quantitative_Backtesting_and_Risk	High	Statistical / Academic	Focuses heavily on avoiding overfitting, assumes valid data ingestion
Quantitative_Signal_Architecture	Medium	Mixed	Blends proven technical signals with theoretical AI overlays
Systems_Level_Reliability_Analysis	Very High	Post-mortem / Forensic	Draws heavily from systemic market crashes; not all apply to retail scale

3. Domain-by-Domain Analysis

Domain A: Storage & Infrastructure

Concepts: Local-first, hybrid DuckDB/SQLite, Litestream WAL replication.

Constraints: Zerodha Kite API rate limits (10 orders/sec, 3 historical requests/sec) enforce asynchronous data daemon requirements.

Verdict: Supported. The combination of embedded transactional (SQLite) and vectorized columnar (DuckDB/Parquet) storage directly mitigates the computational bottlenecks of row-based databases while maintaining ACID compliance.

Domain B: Data Ingestion & Integrity

Concepts: Survivorship bias, corporate action adjustments, data interpolation.

Findings: Zerodha's historical feed is definitively missing candles and is unsafe for unverified backtesting. Upstox automatically applies splits but not dividends.

Verdict: Supported. The requirement for a multi-source validation pipeline (e.g., Angel One + Upstox + Yahoo Finance cross-checks) is mandatory to survive data corruption.

Domain C: Signal Architecture

Concepts: Trend following, momentum, sentiment polarity, mean-reversion.

Findings: Earnings surprises and insider buys carry High signal validity. Analyst upgrades and social media sentiment carry Low validity (mostly noise).

Verdict: Partially Supported. While the hierarchy of signals is empirically backed, the system proposes combining these via AI scoring (e.g., FinBERT), which reintroduces the "black-box" opacity the architecture explicitly attempts to avoid.

Domain D: Execution & Risk Management

Concepts: Non-ergodicity, dynamic margin, concept drift, walk-forward testing.

Findings: Systematic positive feedback loops (e.g., automated volume-based selling) cause liquidity evaporation. AI models that anchor to peak valuations during regime shifts cause catastrophic losses.

Verdict: Supported. Hard kill-switches and probabilistic backtest overfitting (PBO) calculations are absolute requirements for live trading.

4. Cross-Document Consensus Matrix

- **Strong Consensus: AI must never control deterministic execution.** Source 1 states AI invites "catastrophic financial ruin" if allowed to route orders. Source 5 warns of fully AI systems blowing up live if market dynamics shift. Source 6 demonstrates autonomous execution minimizes human intervention but accelerates machine-speed failures.
- **Strong Consensus: Data hygiene is the primary bottleneck.** Source 3 highlights the prevalence of missing intraday candles in Indian broker APIs. Source 4 warns that survivorship bias and unadjusted splits artificially inflate Sharpe ratios.
- **False Consensus: Social Media Sentiment is useful.** While heavily hyped in general knowledge, Source 2 explicitly proves social media adds almost zero predictive power and is a low-reliability signal.

5. Cross-Document Contradiction Matrix

- **Claim A:** "Fully AI (ML/Deep Learning) Systems... generate signals or trades directly... [and] may outperform simple rule systems in turbulent environments".
- **Claim B:** "Execution must remain a strictly deterministic, human-gated process." AI should only handle semantic reasoning.
- **Supporting Sources:** Source 5 vs. Source 1.
- **Evidence Strength:** Claim B is much stronger (95/100) due to strict SEBI regulations and systemic failure case studies (Knight Capital).
- **Which claim is stronger:** Claim B.

- **Remaining Uncertainty:** How AI-generated sentiment scores are mathematically weighted into deterministic execution logic without violating the deterministic barrier.

6. Claim Lineage Matrix

- **Claim:** Real-world trading systems must use a hybrid embedded database (DuckDB + SQLite) rather than vector databases for quantitative backtesting.
- **Supporting Sources:** Source 1.
- **Supporting Evidence:** 95% of swing trading data is structured time-series (OHLCV). Vector databases are built for semantic search, not columnar mathematical aggregations. DuckDB utilizes zero-copy architecture against SQLite WAL.
- **Assumptions:** File-system I/O is fast enough to support DuckDB Parquet scans locally without memory overflow.
- **Confidence:** High (90/100).
- **Validation Required:** Stress-test DuckDB querying multi-year, partitioned 1-minute Parquet tick data against RAM limits.

7. Top 25 Highest Confidence Findings

1. Automated deployment mismatch causes systemic execution failure (Knight Capital).
2. Zerodha historical API data is structurally incomplete and unsuitable as a sole backtesting source.
3. Earnings surprises produce statistically significant post-announcement drift.
4. Insider buying announces reliable bullish sentiment; insider selling does not.
5. Analyst upgrades/downgrades provide near-zero abnormal returns independent of the underlying news.
6. Slippage and transaction costs frequently destroy theoretical backtest alpha.
7. Unadjusted stock splits corrupt technical indicators and backtest validity.
8. Walk-forward cross-validation prevents chronological data leaks ("look-ahead bias") better than randomized k-fold CV.
9. LLM Autonomous Agents fail at deterministic chronological sorting and binary math.
10. FastMCP eliminates network boundaries by serving tool calls via ASGI alongside FastAPI.
11. Dynamic delta hedging creates positive feedback loops that accelerate market crashes.
12. "Auto-Coder" AI backtesters generate massive survivorship bias and curve-fitting.
13. Upstox Uplink API provides up to 20 years of split-adjusted daily data.
14. SEBI regulations mandate static IP, OAuth, and strict rate limiting (10 orders/sec).
15. Market non-ergodicity means historical correlation matrices break down during macro shocks.
16. Zillow Offers collapsed because ML models anchored to peak transactional data during a cooling regime.
17. Sector rotation strategies historically outperform buy-and-hold ~70% of the time when aligned with a broad market trend filter.
18. Moving averages are lagging indicators and fail systematically in sideways/choppy markets.
19. Volume spikes correctly validate price breakouts (adding ~0.8-1.2% premium).
20. Extreme synthetic leverage hidden across multiple prime brokers bypasses standard risk constraints (Archegos).
21. Real-time data feed latency causes electronic market makers to withdraw liquidity, causing Flash Crashes.
22. Bypassing human-in-the-loop controls for AI pricing leads to toxic asset accumulation.

23. A t-statistic > 3.0 or a rigorously deflated Sharpe ratio is required to prove a statistical edge in modern quantitative finance.
24. SQLite is computationally inadequate for time-series aggregations due to its row-oriented engine.
25. Deprecated code left in production binaries represents a massive unquantifiable risk.

8. Top 25 Weakly Supported Findings (Low Confidence)

1. AI sentiment models like FinBERT consistently outperform technicals across all market regimes. (Requires live OOS proof).
2. Promoter share buying is a strong signal. (Evidence shows it often precedes lower returns).
3. Social media sentiment reliably predicts short-term equity swings.
4. FII/DII flow data in India has immediate predictive swing value. (Flows often lag prices).
5. Partially Supported: Retail TV punditry contains actionable alpha.
6. Angel One SmartAPI is 100% free of intraday data gaps. (Community reports note intermittency).
7. Fully AI execution grids can adapt perfectly to 2024 data after training on 2023 data.
8. Yahoo Finance Adjusted Close is perfectly reliable for dividend adjustments. (Occasional mis-adjustments noted).
9. Local SLM (Llama 3.1 8B) can accurately perform complex sentiment tagging on highly nuanced SEBI filings without hallucination.
10. Software reliability precisely mirrors hardware bathtub curves.
11. Equal-weighting signals is superior to optimized risk-parity weighting. (Depends entirely on constraint settings).
12. Institutional fund flows accurately predict 7-day out moves. (Mostly concurrent).
13. LLMs can safely write dynamic SQL queries via MCP against DuckDB. (Extreme hallucination/OOM risk).
14. Browser MCP agents can reliably scrape DOM-heavy financial news sites without breaking.
15. Retail swing strategies can survive purely on 1-minute chart data. (Tick execution latency matters).
16. AI models trained to execute based on textual earnings call transcripts can decipher Indian-specific corporate euphemisms.
17. A 95% confidence interval ($t=2.0$) is sufficient to deploy a trading strategy.
18. Litestream can recover SQLite state to the exact millisecond during a hard disk write failure.
19. Upstox's historical split adjustments handle complex de-mergers perfectly.
20. HMM (Hidden Markov Models) will correctly identify a high-volatility regime before the initial drawdown.
21. "Power Peg" execution loops are impossible in modern brokers.
22. SEBI API regulations will remain static post-2026.
23. Head-and-shoulders chart patterns have an isolated 1-3% statistical edge independent of volume.
24. Vector databases are required to store quantitative market data for AI applications.
25. Small language models can process raw HTML SEC/NSE filings locally without context-window degradation.

9. Top 25 Hidden Assumptions

1. Data Assumption: Exchanges (NSE/BSE) will not retroactively erase or cancel valid trades during a clearing member default (LME Nickel crisis).
2. Infrastructure Assumption: S3/Litestream asynchronous backups will not encounter network dropouts during local PC crashes.

3. Market Assumption: Sovereign credit spreads and developed markets will never face simultaneous, non-ergodic defaults (LTCM).
4. Operational Assumption: An automated data pipeline fetching via cron jobs won't overlap and deadlock if the API latency spikes.
5. Statistical Assumption: Walk-forward out-of-sample data accurately mimics future market regimes.
6. Liquidity Assumption: E-mini or Nifty futures order book depth will remain constant during a 9% intraday drop.
7. Data Assumption: Zero-volume candles in an intraday feed mean no trades occurred, rather than a dropped websocket packet.
8. Behavioral Assumption: Developers will always properly purge deprecated code blocks rather than just unlinking configuration flags.
9. Regulatory Assumption: Brokers will allow high-frequency sequential API polling without shadow-banning IPs.
10. Statistical Assumption: Continuous price paths exist (ignoring gap-downs over weekends).
11. Infrastructure Assumption: Python ASGI servers will not block the event loop when serving simultaneous FastMCP and WebSocket tasks.
12. Market Assumption: Retail stop-loss orders will execute at the trigger price during a flash crash.
13. Model Assumption: Sinusoidal waves accurately capture market seasonality without breaking during macro shocks.
14. Risk Assumption: Option volatility smiles remain static under stress, rather than skewing aggressively.
15. Behavioral Assumption: Risk managers will enforce quantitative limits rather than manually overriding them for high-fee clients.
16. Data Assumption: Parquet file partitioning by year/month is optimal for all backtesting query planners in DuckDB.
17. Model Assumption: Quantile regression uncertainty bands are sufficient risk limiters even if they are only displayed visually.
18. Infrastructure Assumption: The local SQLite WAL file won't grow infinitely if the S3 upload stream hangs.
19. Market Assumption: Multiple prime brokers can liquidate identical collateral without causing a correlated fire sale.
20. Statistical Assumption: Market anomalies identified by an auto-encoder are noise rather than true structural shifts.
21. Execution Assumption: 15-20 minute delays on Yahoo Finance data will not materially affect end-of-day signal calculation.
22. Compliance Assumption: OAuth tokens can be automatically refreshed without manual two-factor authentication intervention daily.
23. Data Assumption: Delisted stocks are not necessary for a valid backtest (Ignoring survivorship bias).
24. Model Assumption: NLP sentiment models (FinBERT) trained on US markets effectively map to Indian corporate disclosures.
25. Operational Assumption: A solo operator will correctly diagnose an HTTP 429 error vs a shadow IP ban.

10. Top 25 Failure Modes

1. Deploying system binaries to some but not all production servers (Uncoordinated Deployment).
2. Order-routing systems running continuous loops without parent-order balance checks.
3. Reusing a configuration flag address space that reactivates a deprecated legacy module.
4. Configuring position limits as passive dashboard alerts rather than hard system halts.

5. Operators silencing critical alert channels due to high-volume alert fatigue.
6. Rolling back a deployment without validating server configurations, spreading old bugs.
7. Dynamic hedging algorithms trend-following by selling assets into declining markets.
8. HFT algorithms mistakenly interpreting runaway execution loops as fundamental volume news.
9. S&P futures backwardation triggering index arbitrageurs to dump cash equities.
10. ML models anchoring to peak historical transaction data during a sudden market downturn.
11. Execution algorithms suppressing uncertainty quantile bounds to maintain volume.
12. Denoising autoencoders masking real market anomalies from risk systems.
13. Bypassing human-in-the-loop safeguards to pursue aggressive automated volume growth.
14. Counterparties hiding massive concentrated exposure by splitting swaps across multiple brokers.
15. Delaying the implementation of dynamic margining to protect short-term trading revenues.
16. Risk managers repeatedly inflating bespoke scenario limits for prestigious clients.
17. Returning billions in variation margin immediately prior to a volatility default.
18. Market spreads failing to converge because of non-ergodic macroeconomic defaults.
19. Multi-broker simultaneous collateral fire sales collapsing underlying asset values.
20. Price-agnostic execution algorithms accelerating selling based on false volume spikes.
21. Automated market makers withdrawing liquidity due to microsecond data feed lags.
22. Exchange Stop-Logic completely trapping unexecuted buy-side orders.
23. Exchanges retroactively erasing valid trades to protect clearinghouse solvency.
24. Incorrect pricing data triggering infinite downstream processing loops (Content-to-Timing failure).
25. Replacing active, in-person risk committees with passive, rubber-stamp email approvals.

11. Top 25 Validation Requirements

1. Mandate automated cluster-wide binary hash verification prior to live routing.
2. Audit all configuration flags to ensure zero reuse of deprecated memory addresses.
3. Hard-code continuous parent-order balance checks directly into execution loops.
4. Enforce position limits via hard API disconnections, not passive alerts.
5. Implement dedicated runbooks for PagerDuty/critical alerts to prevent fatigue.
6. Deploy automated sanity and chaos tests across all nodes before production routing.
7. Hard-code circuit breakers that detect and halt trend-following dynamic hedging cycles.
8. Monitor volume spikes for "informational cascades" and halt non-transparent selling.
9. Track Population Stability Index (PSI) to measure concept drift between live and training data.
10. Program execution algorithms to automatically reduce bid size when quantile uncertainty bands widen.
11. Enforce mandatory multi-signature human approval for algorithmic pricing limit overrides.
12. Audit Total Return Swap margins to match standard prime brokerage initial margin limits.
13. Force the immediate transition of highly leveraged accounts into dynamic margining systems.
14. Hard-code caps on bespoke scenario limits to prevent risk manager inflation.
15. Restrict the release of variation margin during periods of elevated VIX or market stress.
16. Stress-test convergence arbitrage models explicitly against historical correlation breakdowns.
17. Hard-limit execution algorithms from exceeding a strict maximum percentage of the trailing 1-minute volume.

18. Disconnect execution algorithms instantly if consolidated tape latency exceeds predefined microsecond thresholds.
19. Continuously track buy-side order book depth and halt market orders if depth evaporates.
20. Require capital buffers specifically designed to absorb retroactive exchange trade cancellations.
21. Demand a minimum of 90% branch coverage (not just statement coverage) in backtesting modules.
22. Exclude the personal reputation of founders from quantitative credit risk allocation models.
23. Maintain absolute initial margin floors during prolonged bull markets to prevent procyclical compression.
24. Ensure data continuity: explicitly verify that intraday minute feeds equal expected daily lengths (no dropped candles).
25. Cross-verify OHLCV metrics between at least two independent data providers before running signal logic.

12. Blind Spot Inventory

Risk Level	Blind Spot	Mechanism	Under-Tested Edge Case
High	Multi-Broker Leverage Opacity	APIs only track localized risk.	Cross-platform synthetic long accumulation
High	Deprecated Code Reactivation	Unused code left in Python binaries.	Server state mismatch triggering dead loops
High	Uncertainty Decoupling	AI logs wide variance but trades anyway.	Rapid regime shifts ignored by fixed position sizers
Medium	Data Tape Latency Delays	Assuming near-RT WebSockets are instant.	Latency spikes trigger false HFT volume cascades
Medium	Exchange Trade Erasure	Clearinghouses cancel trades to survive.	Unhedged exposure due to retroactively voided legs
Low	Sinusoidal Wave Rigidity	Time-series math forces smooth waves.	Structural economic shifts breaking seasonality

13. Missing Research Inventory

- **Non-Ergodic Daily VaR Models:** The corpus acknowledges non-ergodicity but fails to provide a mathematical framework for calculating daily Value-at-Risk under non-ergodic conditions.
- **Indian Multi-Broker Synthetic Aggregation:** How an algorithmic system specifically manages exposure across multiple Indian discount brokers (e.g., Zerodha + Upstox) simultaneously to avoid aggregate margin breaches.
- **Execution Logic of AI Sentiment Weights:** Exact mathematical formulas converting semantic FinBERT scores (-1 to 1) into localized position sizing (Kelly fractions) without violating the deterministic execution boundaries.
- **Cross-Market Agentic Feedback:** The systemic effect if hundreds of independent retail traders deploy this exact FastMCP local-first architecture on the same NSE tickers.

14. Corpus Coverage Audit

- **Over-Researched Areas:** Broad systemic failure case studies (Knight Capital, LTCM, Zillow). Data ingestion flaws regarding Yahoo Finance and Zerodha. The perils of backtest overfitting and data-snooping.
- **Under-Researched Areas:** The precise cloud deployment topology (Docker/Kubernetes) needed when transitioning from Stage 2 (VM) to Stage 4 (SaaS). Concrete methods for adjusting Options Greeks during dynamic hedging feedback loops.

- **Unsupported Areas:** Live, out-of-sample statistical proof that combining DuckDB, SQLite, and Claude 3.5 via FastMCP actually executes within the 10 orders/sec Zerodha limit successfully without ASGI event loop blocking.

15. Evidence Quality Matrix

- **Systemic Failure Analysis (Knight/LTCM):** Tier 1/2. Highly verified via SEC/PRA reports.
- **Data API Reliability (Upstox/Zerodha):** Tier 3. Relies on practitioner testing and developer forum reports. Subject to silent API updates.
- **Quantitative Signal Validation (Momentum/Earnings):** Tier 2. Backed by major academic literature (Jegadeesh/Titman, NBER).

16. Confidence Matrix

- **High-Velocity Operational Deployment Risk:** 95/100 (High Confidence).
- **Machine Learning Concept Drift (Zillow):** 88/100 (Medium Confidence).
- **High-Frequency Microstructure Crashes:** 92/100 (High Confidence).
- **AI Direct Execution Safety:** 10/100 (Extremely Low Confidence - Actively Contradicted).

17. Unknowns Requiring Future Research

- **Microstructure of Trade Cancellations:** If the NSE clears trades that are later voided due to broker defaults, how does the local SQLite database cleanly revert local WAL logs without corrupting the backtest engine?
- **Ergodic Boundary Definers:** What strict mathematical indicators (e.g., specific volatility clustering metrics) prove an equity has exited an ergodic state and entered an unpredictable tail-risk regime?

18. ADR Candidate Inventory

- **ADR-001: Mandatory Hash Verification:** All production clusters must execute automated binary hash checks before executing code to prevent Knight Capital-style deployments.
- **ADR-002: Dynamic Margin Subsystems:** Execution sizing must auto-scale against portfolio illiquidity; static initial margins are disallowed.
- **ADR-003: Model Uncertainty Integration:** If AI/ML modules output wide quantile regression bands, deterministic execution logic must automatically scale down or halt orders (preventing Zillow Offers failure).
- **ADR-004: Strict FastMCP Boundary Execution:** LLMs may only generate JSON reasoning payloads via FastMCP. Direct execution of `kite.place_order()` by any LLM is strictly prohibited.

19. Architecture-Relevant Findings

- **Storage:** The system MUST utilize an embedded zero-copy architecture (DuckDB scanning Parquet, attaching SQLite via `sqlite_scanner`) to satisfy local-first and high-speed requirements.
- **Data Flow:** Market data MUST be stored in Hive-partitioned Parquet files (Year/Month) for analytical viability; unstructured JSON lakes are prohibited.
- **LLM Routing:** Complex reasoning MUST route to Frontier models (Claude 3.5/Gemini 1.5), routine logic to Fast models (GPT-4o-mini), and PII/Sanitization to local edge SLMs (Qwen 2.5).
- **Resiliency:** The system MUST implement deterministic exponential backoff in the ingestion layer to handle Zerodha HTTP 429 errors safely.

20. Final Research Verdict

The uploaded corpus presents a highly defensive, skeptical, and robust framework for quantitative trading. It accurately identifies that most retail and institutional capital is destroyed not by poor signal generation, but by hidden technical debt, unverified API data, and catastrophic operational feedback loops.

The architecture's mandate to isolate AI exclusively to the "Research/Cognitive" domain while locking mathematical execution, data storage, and order routing in deterministic, embedded SQL/Python environments is the strongest and most validated claim in the corpus. Attempting to build an "Auto-Coder" or fully autonomous AI trading bot within the constraints of Indian Equities (SEBI/Zerodha) will definitively trigger catastrophic failure.

21. "What Would Cause This Entire System To Fail?" Report

Systemic Failure Path:

Extreme Sovereign Macro Shock → Volatility Spikes → Zerodha WebSockets Drop Packets → Deterministic Engine Misinterprets Lags as Zero-Volume Pauses → Claude 3.5 Anchors to Outdated Sentiment (Concept Drift) → FastMCP JSON Processes Order → Local SQLite WAL Locks via Blocked S3 Upload → Execution Circuit Times Out → Runaway Order Routes to Empty Order Book → 15% Execution Slippage Destroys Strategy Capital Buffers

22. "What Evidence Is Still Missing?" Report

- **API Stability Data:** Empirical latency and uptime logs for the Upstox Uplink and Angel One SmartAPI during extreme market events (e.g., election days, budget days).
- **LLM Hallucination Rates on FastMCP:** Specific failure rates for Claude 3.5/Gemini 1.5 returning invalid schema payloads when queried at high frequency over long context windows.
- **DuckDB/SQLite Concurrency Limits:** Benchmarks detailing the exact memory load when DuckDB joins a 50GB Parquet data lake with a live, writing SQLite database locally.

23. "What Assumptions Must Be Tested Before Planning?" Report

- **Test the Broker Rate Limit Thresholds:** Intentionally trigger Zerodha's 429 HTTP errors in a sandbox to validate the deterministic exponential backoff and recovery logic.
- **Test the SQLite WAL S3 Replication:** Force a hardware power-off during a simulated high-volume SQLite transaction write. Attempt to recover the database perfectly from the Litestream S3 bucket upon reboot.
- **Test the Data Validation Pipeline:** Inject synthetic anomalies (e.g., extreme high/low prices, missing minute bars) into the Parquet ingestion engine to verify that the Pandas forward-fill algorithms and outlier detectors catch and isolate the corruption.
- **Test the FastMCP Execution Boundary:** Attempt to prompt-inject the LLM to execute a trade directly, ensuring the ASGI/FastAPI topology physically blocks the agent from accessing the execution endpoints.